

NumPy : Introduction

Python List vs. NumPy Array

A **Python list** and a **NumPy array** may look similar

1	0
2	1
3	2
5	3
7	4

`my_list`

1	0
2	1
3	2
5	3
7	4

`my_numpy_array`

A **NumPy array** is far superior for manipulating large data sets

Python List vs. NumPy Array (Contd.)

NumPy arrays are stored at one continuous place in memory



Python List vs. NumPy Array (Contd.)



Python lists are **NOT** stored at one continuous place in memory



Locality of reference is a significant reason why processing data in NumPy Arrays is faster than processing data in Python lists

Python List vs. NumPy Array (Contd.)

```
my_list = [1,2,3,5,7]
```

list

Attributes

Methods



1

0

2

1

3

2

5

3

```
my_numpy_array = np.array(my_list)
```

numpy.ndarray

Attributes

Methods



1

0

2

1

3

2

5

3

Python List vs. NumPy Array (Contd.)

List	NumPy
Basic data structure	Specialized data structure for numerical data
Can store elements of any data type	Typically stores elements of the same data type
Created using square brackets <code>[]</code>	Created using NumPy functions or arrays
Example: <code>my_list = [1, 1, 3]</code>	Example: <pre>import numpy as np a1 = np.array([1, 2])</pre>
Slower for numerical operations	Designed for fast numerical operations
More memory consumption	Less memory consumption

Creation of NumPy Array

- Creating a 1-D Array

An array with only single row of elements is called 1-D array. Let us try to create a 1-D array from a list which contains numbers as well as strings.

```
>>> array2 = np.array([5,-7.4,'a',7.2])
>>> array2
```

- Creating a 2-D Array

We can create a two dimensional (2-D) arrays by passing nested lists to the array() function.

Example 6.1

```
>>> array3 = np.array([[2.4,3],
                       [4.91,7],[0,-1]])
>>> array3
array([[ 2.4,  3. ],
       [ 4.91,  7. ],
       [ 0. , -1. ]])
```

Attributes of NumPy Array

- i) `ndarray.ndim`: gives the number of dimensions of the array as an integer value. Arrays can be 1-D, 2-D or n-D. In this chapter, we shall focus on 1-D and 2-D arrays only. NumPy calls the dimensions as axes (plural of axis). Thus, a 2-D array has two axes. The row-axis is called axis-0 and the column-axis is called axis-1. The number of axes is also called the array's rank.

Example 6.2

```
>>> array1.ndim
1
>>> array3.ndim
2
```

- ii) `ndarray.shape`: It gives the sequence of integers indicating the size of the array for each dimension.

Example 6.3

```
# array1 is 1D-array, there is nothing
# after , in sequence
>>> array1.shape
(3,)
>>> array2.shape
(4,)
>>> array3.shape
(3, 2)
```


Attributes of NumPy Array

- iii) `ndarray.size`: It gives the total number of elements of the array. This is equal to the product of the elements of shape.

Example 6.4

```
>>> array1.size
3
>>> array3.size
6
```

- iv) `ndarray.dtype`: is the data type of the elements of the array. All the elements of an array are of same data type. Common data types are `int32`, `int64`, `float32`, `float64`, `U32`, etc.

Example 6.5

```
>>> array1.dtype
dtype('int32')
>>> array2.dtype
dtype('<U32>')
>>> array3.dtype
dtype('float64')
```

- v) `ndarray.itemsize`: It specifies the size in bytes of each element of the array. Data type `int32` and `float32` means each element of the array occupies 32 bits in memory. 8 bits form a byte. Thus, an array of elements of type `int32` has `itemsize` $32/8=4$ bytes. Likewise, `int64/float64` means each item has `itemsize` $64/8=8$ bytes.

Example 6.6

```
>>> array1.itemsize
4          # memory allocated to integer
>>> array2.itemsize
128        # memory allocated to string
>>> array3.itemsize
8          #memory allocated to float type
```

Other Way of creating NumPy Array

```
>>> array5 = np.zeros((3,4))
>>> array5
array([[0., 0., 0., 0.],
       [0., 0., 0., 0.],
       [0., 0., 0., 0.]])
```

```
>>> array6 = np.ones((3,2))
>>> array6
array([[1., 1.],
       [1., 1.],
       [1., 1.]])
```

Other Way of creating NumPy Array

```
>>> array7 = np.arange(6)
# an array of 6 elements is created with
# start value 0 and step size 1
>>> array7
array([0, 1, 2, 3, 4, 5])
# Creating an array with start value -2, end
# value 24 and step size 4
>>> array8 = np.arange( -2, 24, 4 )
>>> array8
array([-2,  2,  6, 10, 14, 18, 22])
```

NumPy Array : Indexing

```
# accesses the element in the 1st row in  
# the 3rd column  
>>> marks[0,2]  
56  
>>> marks [0,4]  
index Out of Bound "Index Error". Index 4  
is out of bounds for axis with size 3
```

NumPy Array : Slicing

```
>>> array8
array([-2,  2,  6, 10, 14, 18, 22])

# excludes the value at the end index
>>> array8[3:5]
array([10, 14])

# reverse the array
>>> array8[ : : -1]
array([22, 18, 14, 10,  6,  2, -2])
```

```
>>> array9 = np.array([[ -7,  0, 10, 20],
                        [ -5,  1, 40, 200],
                        [ -1,  1,  4, 30]])

# access all the elements in the 3rd column
>>> array9[0:3,2]
array([10, 40,  4])
```

```
# access elements of 2nd and 3rd row from 1st
# and 2nd column
>>> array9[1:3,0:2]
array([[ -5,  1],
       [ -1,  1]])
```

NumPy Array Reshaping

```
>>> array3 = np.arange(10,22)
>>> array3
array([10, 11, 12, 13, 14, 15, 16, 17, 18,
       19, 20, 21])
```

```
>>> array3.reshape(3,4)
array([[10, 11, 12, 13],
       [14, 15, 16, 17],
       [18, 19, 20, 21]])
```

```
>>> array3.reshape(2,6)
array([[10, 11, 12, 13, 14, 15],
       [16, 17, 18, 19, 20, 21]])
```

NumPy Array Sorting

```
>>> array4
array([[ 10,  -7,   0,  20],
       [ -5,   1, 200,  40],
       [ 30,   1,  -1,   4]])

#default is row-wise sorting
>>> array4.sort()
>>> array4
array([[ -7,   0,  10,  20],
       [ -5,   1,  40, 200],
       [ -1,   1,   4,  30]])
>>> array5 = np.array([[10,-7,0, 20],
                       [-5,1,200,40],[30,1,-1,4]])

#axis =0 means column-wise sorting
>>> array5.sort(axis=0)
>>> array5
array([[ -5,  -7,  -1,   4],
       [ 10,   1,   0,  20],
       [ 30,   1, 200,  40]])
```

Splitting Array

```
>>> array4
array([[ 10,  -7,   0,  20],
       [ -5,   1, 200,  40],
       [ 30,   1,  -1,   4],
       [  1,   2,   0,   4],
       [  0,   1,   0,   2]])

# [1,3] indicate the row indices on which
# to split the array
>>> first, second, third = numpy.split(array4,
                                       [1, 3])

# array4 is split on the first row and
# stored on the sub-array first
>>> first
array([[10, -7,  0, 20]])

# array4 is split after the first row and
# upto the third row and stored on the
# sub-array second
>>> second
array([[ -5,   1, 200,  40],
       [ 30,   1,  -1,   4]])

# the remaining rows of array4 are stored
# on the sub-array third
>>> third
array([[1, 2, 0, 4],
       [0, 1, 0, 2]])
```


Splitting Array

```
#[1, 2], axis=1 give the columns indices
#along which to split
>>> firstc, secondc, thirdc =numpy split(array4,
[1, 2], axis=1)
>>> firstc
array([[10],
       [-5],
       [30],
       [ 1],
       [ 0]])

>>> secondc
array([[ -7],
       [ 1],
       [ 1],
       [ 2],
       [ 1]])

>>> thirdc
array([[ 0, 20],
       [200, 40],
       [-1,  4],
       [ 0,  4],
       [ 0,  2]])
```

Splitting Array

```
# 2nd parameter 2 implies array is to be
# split in 2 equal parts axis=1 along the
# column axis
>>> firsthalf, secondhalf = np.split(array4, 2,
axis=1)
>>> firsthalf
array([[10, -7],
       [-5,  1],
       [30,  1],
       [ 1,  2],
       [ 0,  1]])

>>> secondhalf
array([[ 0, 20],
       [200, 40],
       [-1,  4],
       [ 0,  4],
       [ 0,  2]])
```

Statistical Operations

Let us consider two arrays:

```
>>> arrayA = np.array([1,0,2,-3,6,8,4,7])  
>>> arrayB = np.array([[3,6],[4,2]])
```

1. The `max()` function finds the maximum element from an array.

```
# max element form the whole 1-D array  
>>> arrayA.max()  
8  
# max element form the whole 2-D array  
>>> arrayB.max()  
6  
# if axis=1, it gives column wise maximum  
>>> arrayB.max(axis=1)  
array([6, 4])  
# if axis=0, it gives row wise maximum  
>>> arrayB.max(axis=0)  
array([4, 6])
```

2. The `min()` function finds the minimum element from an array.

```
>>> arrayA.min()  
-3  
>>> arrayB.min()  
2  
>>> arrayB.min(axis=0)  
array([3, 2])
```

Statistical Operations

3. The `sum()` function finds the sum of all elements of an array.

```
>>> arrayA.sum()
25
>>> arrayB.sum()
15
#axis is used to specify the dimension
#on which sum is to be made. Here axis = 1
#means the sum of elements on the first row
>>> arrayB.sum(axis=1)
array([9, 6])
```

4. The `mean()` function finds the average of elements of the array.

```
>>> arrayA.mean()
3.125
>>> arrayB.mean()
3.75
>>> arrayB.mean(axis=0)
array([3.5, 4. ])

>>> arrayB.mean(axis=1)
array([4.5, 3. ])
```

5. The `std()` function is used to find standard deviation of an array of elements.

```
>>> arrayA.std()
3.550968177835448
```

Load Data from TextFile

RollNo	Marks1	Marks2	Marks3
1,	36,	18,	57
2,	22,	23,	45
3,	43,	51,	37
4,	41,	40,	60
5,	13,	18,	37

```
>>> studentdata = np.loadtxt('C:/NCERT/  
data.txt', skiprows=1, delimiter=',',  
dtype = int)
```

```
>>> studentdata  
array([[ 1, 36, 18, 57],  
       [ 2, 22, 23, 45],  
       [ 3, 43, 51, 37],  
       [ 4, 41, 40, 60],  
       [ 5, 13, 18, 27]])
```

genfromtxt()

`genfromtxt()` is another function in NumPy to load data from files. As compared to `loadtxt()`, `genfromtxt()` can also handle missing values in the data file. Let us look at the following file `dataMissing.txt` with some missing values and some non-numeric data:

RollNo	Marks1	Marks2	Marks3
1,	36,	18,	57
2,	ab,	23,	45
3,	43,	51,	
4,	41,	40,	60
5,	13,	18,	27

```
>>> dataarray = np.genfromtxt('C:/NCERT/
dataMissing.txt', skip_header=1,
delimiter = ',')
```

```
>>> dataarray
array([[ 1., 36., 18., 57.],
       [ 2., nan, 23., 45.],
       [ 3., 43., 51., nan],
       [ 4., 41., 40., 60.],
       [ 5., 13., 18., 27.]])
```

```
>>> dataarray = np.genfromtxt('C:/NCERT/
dataMissing.txt', skip_header=1,
delimiter=',', filling_values=-999,
dtype = int)
```

```
>>> dataarray
array([[ 1, 36, 18, 57],
       [ 2, -999, 23, 45],
       [ 3, 43, 51, -999],
       [ 4, 41, 40, 60],
       [ 5, 13, 18, 27]])
```

genfromtxt()

```
>>> dataarray = np.genfromtxt('C:/NCERT/  
dataMissing.txt', skip_header=1,  
delimiter=',', filling_values=-999,  
dtype = int)
```

```
>>> dataarray  
array([[ 1,  36,  18,  57],  
       [ 2, -999,  23,  45],  
       [ 3,  43,  51, -999],  
       [ 4,  41,  40,  60],  
       [ 5,  13,  18,  27]])
```